

PROJECT REPORT

The Signal to Trade.

A machine-learning pipeline that reads the market — fundamentals, news, crowd sentiment, and price — and passes its evidence through an LLM gate to decide what to buy, what to avoid, and where to set the stop.



MARKET DATA FEATURES FORECAST SENTIMENT LLM GATE SIZING

SYSTEM **Stock Market ML Automation**
SCOPE **Indian equities (NSE), swing horizon**
UNIVERSE **49 NIFTY-50 stocks**

PREPARED FOR **Kingshuk**
AUDIENCE **Technical + stakeholder**
STATUS **Live · backtest pending**

What this system is

At its core the project answers one question every trading day: *of the stocks we follow, which are worth buying tomorrow, and on what terms?* It does not guess. It assembles four independent readings of each stock — how the business is doing, what the news and the crowd are saying, and where a forecasting ensemble thinks the price is headed — and hands that dossier to a large language model that acts as a disciplined analyst, returning a **BUY / HOLD / AVOID** call with an entry price, a target, a sell day, and a protective stop.

The design bias throughout is capital preservation before activity. The forecasting models never predict a single number; they predict a band of outcomes. The LLM is instructed that outputting zero buys across every stock is often the correct answer. And a separate deterministic layer re-checks every buy the LLM proposes against hard rules learned from trade history, so a good narrative can never override bad arithmetic. This is a system built to say *no* cheaply and *yes* only when the reward genuinely pays for the risk.

4	32	3	q10.q90	16
EVIDENCE CHANNELS	MODEL FEATURES	ENSEMBLE MODELS	FORECAST BAND	PIPELINE STAGES

THE EDGE, IN ONE LINE

Four uncorrelated signals feed one gate, the gate is forced to justify every buy against a costed reward-to-risk test, and a rule engine guarantees the discipline the prompt only asks for. The advantage is not any single model — it is the refusal to act on a thesis that the numbers do not survive.

Seven stages, one decision

Market data flows left to right through a fixed contract. Each stage depends on the one before it; a failure upstream stops the chain rather than letting a half-formed signal reach the trading layer. The LLM gate sits at the centre — everything above it gathers evidence, everything below it turns a decision into a sized, risk-bounded position.

01	Ingestion yfinance OHLCV · quarterly fundamentals · news · Reddit · X
02	Feature engineering 32 technical indicators + cross-sectional / regime features
03	Forecast ensemble Dense · LSTM · Transformer → q10 / q50 / q90 price path
04	Sentiment FinBERT polarity, importance + recency weighted per channel
05	LLM decision gate GLM-4.7 → BUY / HOLD / AVOID with target + stop-loss
06	Entry gates deterministic re-check of momentum, volume, stop, reward:risk
07	Capital allocation vol-targeted Markowitz sizing, conviction-scaled deployment

1 _d	15 _d	Mo.	3
DATA CADENCE	FORECAST PATH USED BY GATE	INCREMENTAL FINE-TUNE	STORAGE SURFACES

Storage is split by purpose: real prices and forecasts live in **Google Sheets**; news, sentiment, fundamentals and the final trade suggestions live in **Firebase / Firestore**; and a local **Parquet** archive retains history for training and drift monitoring. Orchestration, scheduling (GitHub Actions), a FastAPI server, and Docker parity wrap the whole thing.

Reading the market four ways

A price forecast alone is a thin basis for a trade. The system deliberately triangulates, and it keeps the channels *separate* because they behave differently — news lags, Reddit runs contrarian, X is fast and noisy, and a combined average would smear those distinct signals into mush.

Fundamentals & price

- **OHLCV** — daily bars pulled via `yfinance`, appended to the operational sheet, mirrored to a local store for integrity checks.
- **Quarterly fundamentals** — the last four quarters of revenue, net income and related lines per ticker, refreshed weekly and stored one document per company.

News & social sentiment

- **News** — headlines from the last 7 days, article bodies scraped in full, mapped to companies by an alias matcher; macro items flow through as a market-level bucket.
- **Reddit & X** — posts scraped from NIFTY-focused subreddits and X, deduplicated, matched to tickers.
- **FinBERT scoring** — each item is condensed to a polarity in $[-1, 1]$, then aggregated per ticker and day.

NOT EVERY HEADLINE COUNTS THE SAME

Each item's weight is $\text{recency} \times \text{engagement} \times \text{source_quality} \times \text{confidence}$. A viral post cannot drown out the day through log-scaled engagement; trusted outlets outweigh unknown blogs; and mushy, neutral text is discounted. Recency decays on a configurable half-life (default 3 days), so the latest headline dominates and a week-old one barely registers.

A band, never a point

From the engineered features, a three-model PyTorch ensemble produces a multi-horizon *quantile* forecast: for each day ahead it emits the 10th, 50th and 90th percentile of the log-return, so the output is an explicit cone of uncertainty rather than a single deceptive line. Predictions are bounded by the exchange's daily circuit limit, so no horizon can imply a physically impossible move.

Dense

MLP BASELINE

LSTM

SEQUENTIAL MEMORY

Trans.

ATTENTION ENCODER

20

DAY LOOKBACK WINDOW

Features the models see

32 indicators per bar — momentum (RSI, MACD, ROC, Williams %R), trend (SMA / EMA families, ADX), volatility (ATR plus a GARCH-style conditional-vol family), and volume (OBV, VWAP, MFI) — extended with cross-sectional and regime features that place each stock *relative* to the NIFTY index and the India VIX. Forward-return labels for horizons up to 30 days are targets, never inputs, and are stripped before inference.

LEARNING THAT NEVER FORGETS THE WHOLE BOOK

The models are never retrained from scratch. A gated monthly fine-tune warm-starts the saved checkpoints on only the rows they have never seen — triggered by a new calendar month, or early once the median stock accumulates ~30 fresh rows — and atomically overwrites the weights in place. Retraining is incremental by design: cheap, frequent, and never destabilising.

Where evidence becomes a trade

The heart of the system. Each stock's 15-day forecast path, its sentiment snapshot and its recent fundamentals are assembled into one dossier and sent to GLM-4.7 under a strict system prompt. The model is told to think like a portfolio manager — capital preservation first, asymmetric reward second, activity last — and to pass every candidate buy through five gates *in order*. Fail any gate, and the answer is HOLD or AVOID.

GATE	RULE
1 · Trend	Price above its close ~15 days ago and the forecast path ends above today. A bullish forecast on a falling stock is catching a knife.
2 · Entry	Momentum orderly, not climactic — never a steep oversold slide, never a volume/price spike day (those are distribution, not accumulation).
3 · Stop	Stop below recent swing support and at least 2× the typical daily move below entry. A stop inside one day's range is noise, not protection.
4 · Reward	After 0.9% round-trip cost, reward must be $\geq 1.5\times$ the risk. If the maths does not clear, there is no trade.
5 · No chase	Entry within 3% of the latest close.

Confidence is *computed, not felt*: it starts at 0.5 and moves on an explicit rubric (recent price change, forecast shape, sentiment, fundamentals, volatility), and only a score ≥ 0.6 permits a BUY. The model returns a single JSON object — action, buy price, sell day, target, stop-loss, confidence, and a rationale that must cite each gate. Runs are incremental: a stock is only re-sent when its forecast or sentiment actually changes, fingerprinted by a hash.

DISCIPLINE THE PROMPT ASKS FOR, THE CODE GUARANTEES

A deterministic entry_gates layer re-checks every BUY against the same rules in plain arithmetic — momentum-and-forecast agreement, a 1.5× average-volume ceiling, an RSI window, an ATR stop floor, and the costed reward:risk bar. A buy that fails a hard gate is downgraded to HOLD with the reason recorded; a too-tight stop is first widened to the ATR floor and only rejected if the reward no longer clears the bar. The thresholds are drawn from a walk-forward analysis of 668 closed round trips — so the rules are earned, not assumed.

A separate position-review prompt manages *open* trades — letting winners run, trailing stops upward, and cutting broken theses — so the gate governs both entry and exit.

From a call to a position size

A BUY is only the beginning of a decision. The allocation engine turns the discrete calls into concrete rupee sizing using a volatility-targeted Markowitz optimisation, tilted mildly by the LLM's confidence, and subject to guardrails that make a catastrophic drawdown structurally difficult.

- **Cash floor** — a fraction of equity is always retained; the book can never be fully drawn down.
- **Per-name cap** — hard ceiling per stock, shrunk further for volatile names via a volatility target.
- **Conviction-scaled deployment** — how much of the book to deploy is not fixed; weak-but-still-BUY sets deploy only a floor, ramping to full deployment only when the weighted reward:risk is genuinely excellent.
- **No invented capital** — cash never goes negative; a BUY is funded by rotating out of an incumbent only if it beats that holding on reward:risk by a configured edge, otherwise it is downsized to the cash actually available.

STOP - LOSS	ENTRY	TARGET · ≥ 1.5× RISK
-------------	-------	----------------------

Every position the system advises carries this shape — a stop it can survive and a target that, net of costs, pays at least one and a half times what it puts at risk.

Built to run itself

The pipeline is production-shaped, not a notebook. A 16-stage orchestrator runs the daily path with a strict contract, hard gates that abort on failure, and dry-run as the default so no live sheet is ever touched by accident.

Automation surface

- **Scheduled jobs** — GitHub Actions for daily data + prediction, weekly fundamentals, scheduled news/sentiment, and the monthly gated fine-tune.
- **Serving** — a FastAPI server plus a Google-Sheets polling watcher for event-driven runs; Docker Compose for deployment parity.
- **MLOps** — model artifacts versioned and uploaded to GitHub Releases; metadata tracked per run.
- **Tests** — a pytest suite guards the contract; the entry-gate maths is pure and unit-tested independently of any network.

BACKTEST — RUNNING NOW, ARRIVING NEXT

A walk-forward backtest across 2020–2026 is currently in progress and is deliberately left out of this edition. It is the layer that will measure realised P&L, win rate and drawdown, and validate the same allocation engine used in production. This report will be updated with those results once the run completes.